Introduction to Design Patterns (Part 2)

Creational patterns — Advanced Programming

Ayoub FRIHAOUI

ayoub.frihaoui@lecnam.net
Conservatoire national des arts et métiers (CNAM Paris)

Original material © Rezak AZIZ, CNAM Paris.
Source: `https://rezakaziz.github.io/PROG-AVANC/`

Creational patterns

Creational Patterns

Definition

Creational Patterns describe different ways of managing, controlling and structuring **object instantiation**, to reduce coupling and improve design flexibility.

Main creational Patterns

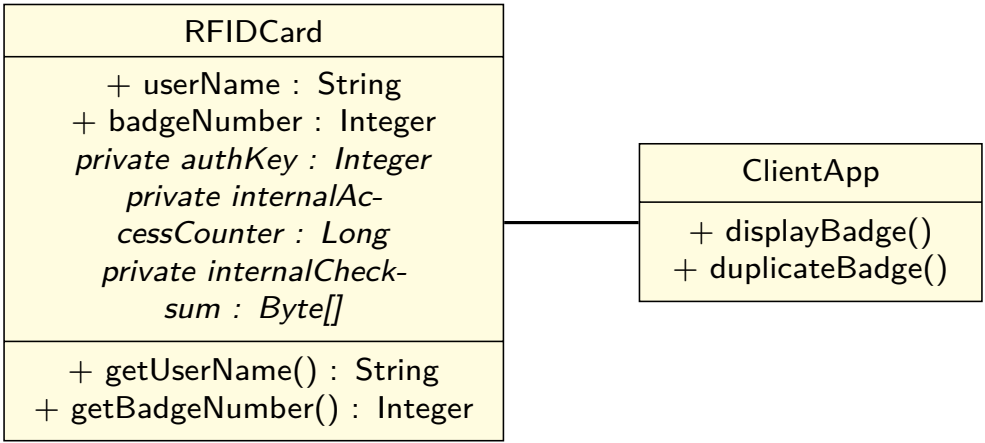
- **Singleton** — guarantees a class has only one instance.
- **Prototype** — creates new objects by cloning an existing object.
- **Factory Method** — delegates creation to a specialised method, redefined by subclasses.
- **Abstract Factory** — creates families of related objects without knowing the concrete classes.
- **Builder** — separates building a complex object from its final representation.

Section 1

Prototype

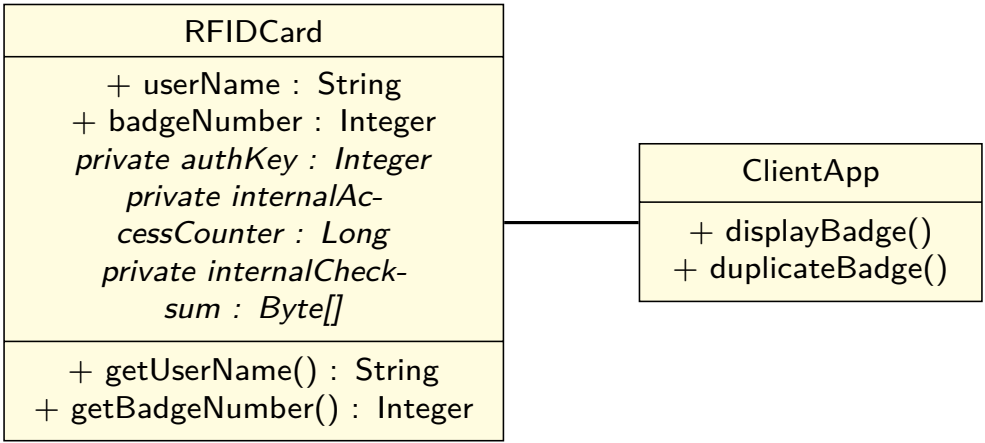
Problem to solve

"How would you copy an object in programming?"



Problem to solve

"How would you copy an object in programming?"



"How would you implement the `duplicateBadge()` method?"

Copying an object from the outside is a problem?

Naive approach

1. Create a new object of the same class;
2. Copy all its attributes one by one.

Question: where is the problem?

- Some attributes are private;
- You must know the **concrete** class;
- Sometimes you only know the *interface*;
- Complex hierarchies $\rightarrow$ hard to rebuild.



General intent

Intent

Prototype is a design pattern that creates new objects from existing objects without making the code depend on their class.

Key idea

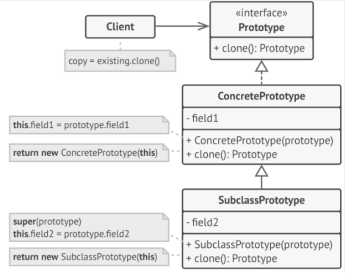
Define a `clone()` method in a common interface and delegate duplication to the object.

Solution

Principle

Define a `clone()` method in a common interface and delegate duplication to the object.

Structure



Roles

- **Prototype**: interface declaring `clone()`;
- **ConcretePrototype**: implements internal cloning;
- **Client**: clones objects without knowing their class.

Prebuilt prototypes

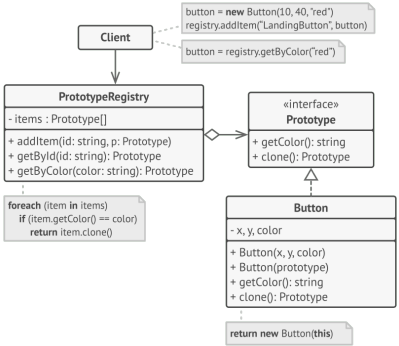
Principle

Store prebuilt prototypes in a hash table:

- key → Prototype;
- `clone()` to get an instance.

Usefulness

- avoid repeatedly creating complex objects;
- provide reusable templates;
- easily substitute a configuration.



Applicability

When to use Prototype?

- The code must not depend on concrete classes;
- Objects are configured through an external interface;
- Many configurations are possible;
- You want to avoid massive subclassing.

Concrete cases

- graphical elements;
- heavy objects to initialise;
- template documents;
- 3D scenes, textures, preconfigured widgets.

Advantages and Drawbacks

Advantages

- Cloning without knowing the concrete class;
- No more repetitive initialisation code;
- Fast creation of complex objects;
- Elegant alternative to subclassing.

Drawbacks

- Cloning objects with circular references is hard;
- Deep cloning can be costly;
- Copy constructors must be well maintained.

Section 2

Factory Method

Problem to solve

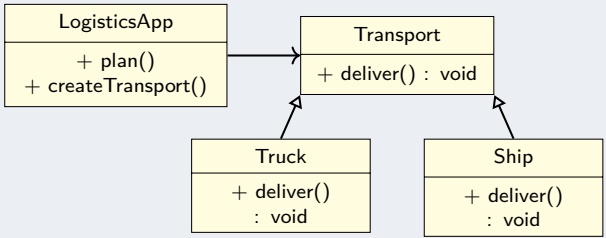
"How to implement the `createTransport` method?"

Naive approach

- `new` scattered all over the code;
- Each new transport forces modifying `LogisticsApp`;
- Multiple conditions:
  - `if (type == truck)...`
  - `if (type == ship)...`

Strong coupling + violation of the open/closed (OCP) principle.

Example



General intent

Intent

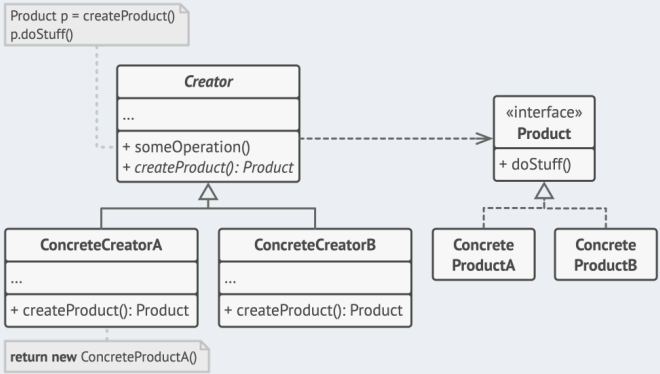
Factory is a creational design pattern that defines an interface for creating objects in a parent class, but delegates the choice of object types to create to subclasses.

Key idea

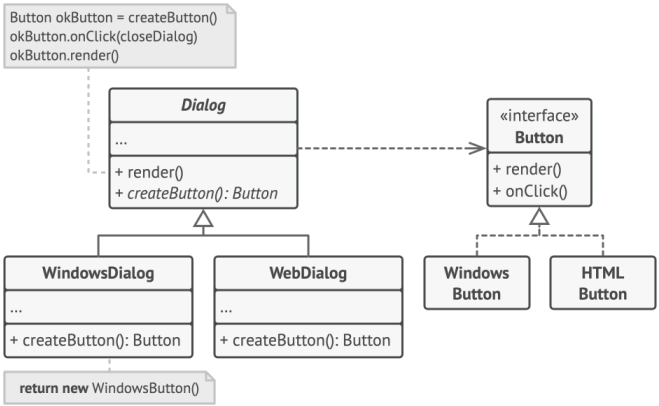
Replace: `new Truck()`
with: `createTransport()`
The subclass decides whether it returns a Truck, a Ship, etc.

Solution

Structure



Example: building a multi-platform application



Applicability

When to use Factory Method?

- The code must not depend on concrete classes;
- You want to add new products easily;
- Behaviour varies with the subclass;
- Frameworks or extensible applications.

Concrete cases

- Multiplatform UI widgets;
- Network connections;
- Plugins and extensions;
- Object pool management.

Advantages and Drawbacks

Advantages

- Creator / concrete-product decoupling;
- Extensible without modifying existing code (OCP);
- Fewer conditions in the code;
- Better code organisation.

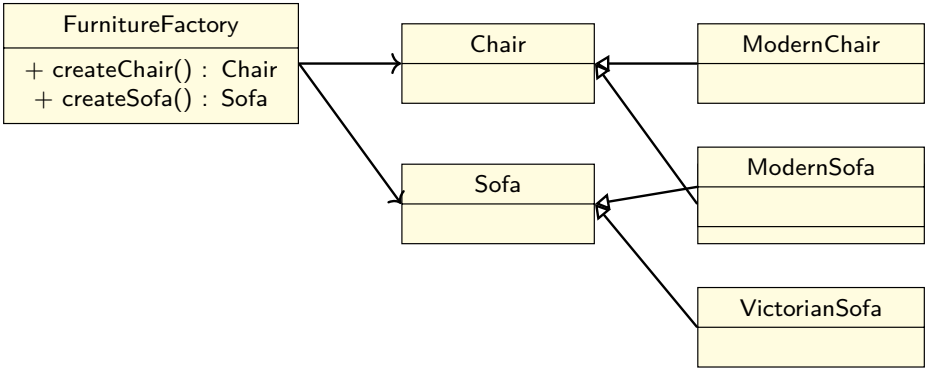
Drawbacks

- More classes to create;
- Initial complexity for beginners;
- Needs a common product interface.

Section 3

Abstract Factory

Problem to solve



"How to guarantee all furniture of one style stays compatible?"

Combining variants by hand is a problem

Naive approach

- Create objects one by one with `new`;
- Mix variants with conditions:
  - `if (style == Modern) new ModernChair();`
  - `else if (style == Victorian) new VictorianChair();`
- Repeat this scheme for Sofa, coffee table, etc.

Where is the problem?

- Risk of mixing styles by mistake;
- Conditions duplicated everywhere in the code;
- Hard to add a new variant without breaking everything.

General intent

Intent

The **Abstract Factory** is a creational pattern that creates **families of related objects** without specifying their concrete class.

Key idea

- The client code only works with interfaces: Chair, Sofa, CoffeeTable, etc.
- A **concrete factory** guarantees all its products belong to the **same variant** (Modern, Victorian, ArtDeco, ...).

Solution

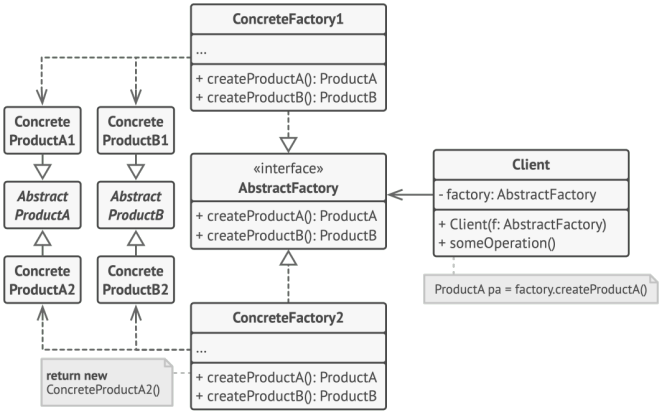
Principle

1. Define **abstract products** (Chair, Sofa, CoffeeTable);
2. Define an **Abstract Factory** that creates each product type;
3. Implement one **Concrete Factory per variant** (ModernFactory, VictorianFactory, ...).

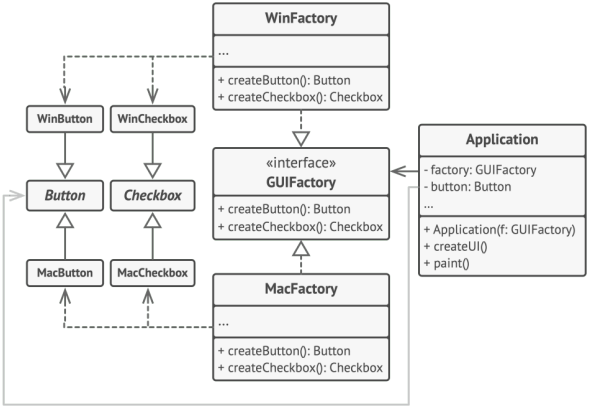
Effect

- All created objects belong to the **same family**;
- They are **compatible** with each other;
- The client code never sees the **concrete classes**.

Structure of the Abstract Factory



Example: building a multi-platform application



Applicability

When to use Abstract Factory?

- You have **families of related objects**;
- You want to guarantee their **compatibility**;
- You want to switch variant at once (theme, skin, graphic style...);
- The code must depend on no concrete class.

Typical cases

- Multiplatform graphical interfaces;
- Themes / skins (light, dark, high-contrast);
- Modern / Victorian / ArtDeco furniture;
- Video games: races, factions, coherent environments.

Advantages and drawbacks

Advantages

- Guaranteed compatibility between products of one family;
- Total decoupling of client and concrete products;
- Centralised variant switch (a single factory);
- Respects the open/closed principle for variants.

Drawbacks

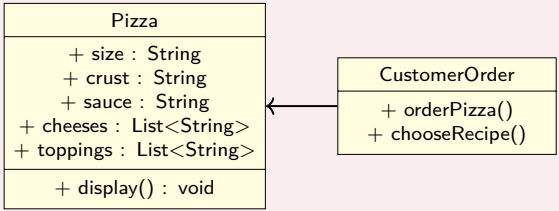
- Many classes (products + factories);
- Adding a new **family of products** means modifying all factories;
- Pattern sometimes seen as “heavy” for small projects.

Section 4

Builder

Problem to solve

Example: building a Pizza



"How to avoid a giant constructor like: `Pizza(size, crust, sauce, cheese1, cheese2, olive, mushrooms, ...)`?"

Classical constructor: too complex

Naive approach

```
new Pizza("Large", "Thin",
    "Tomato",
    ["Mozza", "Parmesan"],
    ["Olives", "Mushrooms",
     "Ham"])
```

→ unreadable, easy to get wrong, hard to maintain.

Naive approach 2

```
Pizza p = new Pizza();
p.setSize("Large");
p.setCrust("Thin");
p.setSauce("Tomato");
...
```

→ readable, easy.

Problem

- Too many parameters;
- Impossible to guarantee consistency;
- Infinite pizza combinations (Margherita, 4 cheeses, Veggie...).

General intent

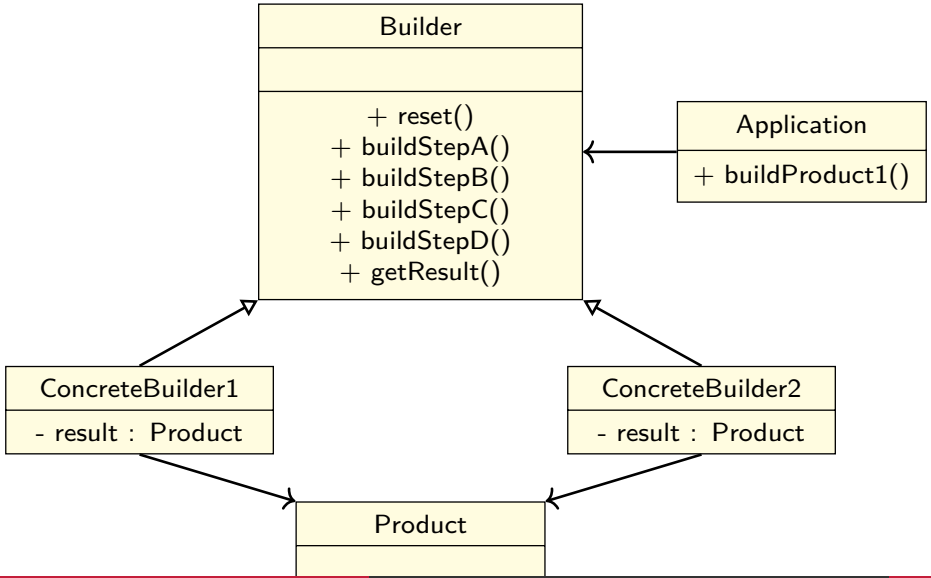
Intent

Builder is a creational design pattern that builds complex objects step by step. It produces different variations or representations of an object using the same construction code.

Key idea

Make the constructor accessible only through a builder, and go through the builder to construct the concrete object.

Structure of the Builder



Applicability

When to use Builder?

- Complex objects (lots of options);
- Progressive step-by-step construction;
- Several possible representations;
- Avoid giant constructors.

Concrete cases

- Building immutable objects;
- Document generation (manual for product);
- Complex SQL queries;
- 3D scenes, graphical interfaces;
- DevOps pipelines.

Advantages and Drawbacks

Advantages

- Flexible, controlled construction;
- Same sequence $\rightarrow$ different products;
- Readable, organised, coherent code;
- Construction / representation decoupling.

Drawbacks

- More classes;
- Heavy setup for small objects;
- Requires clearly defining the steps.